Grado en Ingeniería Informática 2026

Heurística y Optimización

Práctica de Programación Lineal

Laura Parrilla Ruiz - 100522167@alumnos.uc3m.es

Beatriz Esteban González - 100550177@alumnos.uc3m.es



Parte 1: Minimización del coste medio

Descripción del problema

El problema que se presenta es un problema de optimización basado en un pallet cuadrado de lado l, en el que nos piden minimizar el coste de colocación de unas cajas sobre el pallet. Este pallet dispone de l^2 posiciones y debe llenarse completamente con $n=l^2$ cajas, todas del mismo tamaño y peso. Cada caja tiene asociada una prioridad en números. Entonces, aquella que tenga un valor más bajo tendrá mayor prioridad. El pallet, además, tiene un único punto de acceso, junto a la tercera fila. Por esto, las cajas deben colocarse respetando el orden de prioridad dentro de cada columna (caja situada más cerca de la entrada deberá tener mayor prioridad, que es lo mismo a un número menor que las que están delante de ella).

Para resolver este problema se propone un modelo de **Programación Lineal Entera Binaria**, basado en variables de asignación que indican qué caja ocupa cada posición del pallet.

Descripción del modelo

Parámetros y conjuntos

Se definen los siguientes parámetros:

- l: longitud del lado del pallet.
- $n=l^2$: número total de cajas.
- p_k: prioridad numérica de la caja k (valor menor → prioridad mayor).

Se consideran los conjuntos:

$$K = \{1, 2, \ldots, n\}$$

$$I = \{1, 2, \ldots, l\}$$

$$J = \{1, 2, \ldots, l\}$$

Donde:

- K: conjunto de cajas.
- I: conjunto de filas.
- J: conjunto de columnas.

Variables de decisión

Se define una variable (binaria) para cada caja k y posición (i, j):

$$x_{kij} = \begin{cases} 1 \text{ si la caja } k \text{ ocupa la posición } (i, j), \\ 0 \text{ en caso contrario} \end{cases}$$

Su dominio es:

$$x_{kij} \in \{0, 1\} \qquad \forall k \in K, i \in I, j \in J$$

Utilizando esta representación, podemos identificar de forma única la posición asignada a cada caja y formular las restricciones de ocupación y prioridad, todo realizado de manera lineal.

Restricciones

1. Asignación única de cada caja

Cada caja debe ocupar exactamente una posición del pallet:

$$\sum_{i \in I} \sum_{j \in J} x_{kij} = 1 \quad \forall k \in K$$

Con esta restricción, garantizamos que todas las cajas sean asignadas y evitamos que una misma caja aparezca en varias posiciones.

2. Ocupación única de cada posición

Como el pallet debe estar completamente lleno, cada posición debe entonces contener exactamente una caja:

$$\sum_{k \in K} x_{kij} = 1 \qquad \forall i \in I, j \in J$$

Esta restricción nos garantiza que todas las cajas sean asignadas y evita que una misma caja aparezca en varias posiciones.

3. Ordenación de las prioridades

Las cajas deben colocarse respetando su prioridad dentro de cada columna. Puesto que un valor numérico menor representa una prioridad mayor, la caja situada en una fila superior debe tener una prioridad numérica menor o igual que la situada inmediatamente debajo. Entonces, la prioridad de la caja asignada a una posición la expresamos como:

$$\sum_{k \in K} p_k x_{kij}$$

Esta expresión devuelve la prioridad de la caja que ocupa la posición, puesto que únicamente una variable de asignación toma el valor 1. Teniendo esto, la restricción de prioridad es entonces:

$$\sum_{k \in K} p_k x_{k,i+1,j} \leq \sum_{k \in K} p_k x_{kij}$$
$$\forall i \in \{1, \ldots, l - 1\}, \qquad \forall j \in J$$

Esta desigualdad establece que el valor numérico de la prioridad en la fila $i + 1$, más próxima al acceso, debe ser menor o igual que el correspondiente a la fila i.

Formalización de la función objetivo

El objetivo es minimizar el coste medio de disposición de todas las cajas. Siguiendo el enunciado, sabemos que el coste de una caja en posición (i, j) es la suma de las prioridades de todas las cajas que se encuentran entre ella y el acceso, dentro de su misma columna:

$$c_{ij} = \sum_{r=i+1}^{l} p_{rj}$$

Donde p_{rj} representa la prioridad de la caja situada en la fila *r*, columna *j*. Para expresar la función objetivo mediante las variables de decisión, observamos que una caja situada en la fila *i* contribuye al coste de las *i* - 1 cajas que se encuentran detrás de ella, hacia el fondo del pallet. Por tanto, la prioridad aparece exactamente *i* - 1 veces en la suma de costes. Entonces:

$$c_{total} = \sum_{k \in K} \sum_{i \in I} \sum_{j \in J} (i - 1) p_k x_{kij}$$

Dividiendo entre el número total de cajas, se obtiene la función objetivo:

$$\min Z = \frac{1}{n} \sum_{k \in K} \sum_{i \in I} \sum_{j \in J} (i - 1) p_k x_{kij}$$

Parte 2: Montaje volumétrico

Nota: Dado a que los modelos son continuaciones y expansiones de las últimas partes, solo se anotará lo que hayamos añadido o cambiado, el resto de parámetros o restricciones no se mencionan o simplemente no se volverán a explicar.

Descripción del problema

El problema presentado en este apartado es una expansión del apartado anterior, donde consideramos la misma función objetivo y restricciones que en el apartado 1 pero con la dificultad añadida de que ahora nuestros pallets pueden tener niveles. Esos niveles de altura llevan consigo restricciones con tanto el peso que ahora se le atribuye a cada caja como sus respectivas capacidades de carga. Nosotras hemos afrontado esta nueva dificultad cambiando la manera en la que nuestro modelo calcula el coste total y las restricciones para que ahora tengan en cuenta el aspecto 3D del pallet, lo cual se puede ver en la descripción del modelo.

Descripción del modelo

Parámetros y conjuntos

Se definen los siguientes **parámetros**:

- $p_k \in Z^+$: prioridad numérica de la caja $k \in K$(valor menor $\rightarrow$ prioridad mayor).
- $\omega_k \in (R \geq 0)$: peso de la caja $k \in K$
- $\gamma_k \in (R \geq 0)$: capacidad de carga de la caja $k \in K$

Se consideran los conjuntos:

$$K = \{1, 2, \dots, n\} \qquad J = \{1, 2, \dots, l\}$$

$$I = \{1, 2, \dots, l\} \qquad S = \{1, 2, \dots, h\}$$

Donde:

- l: longitud del lado del pallet.
- $n = h \cdot l^2$: número total de cajas.
- h: número máximo de niveles
- K: conjunto de cajas.
- I: conjunto de filas.
- J: conjunto de columnas.
- S: conjunto de niveles

Variable de decisión

La variable (binaria) para cada caja *k* y posición (*i, j, s*):

$$x_{k,i,j,s} = \begin{cases} 1 \text{ si la caja } k \text{ ocupa la posición } (i, j, s), \\ 0 \text{ en caso contrario} \end{cases}$$

Su dominio es:

$$x_{k,i,j,s} \in \{0,\ 1\} \forall k \in K,\ i \in I,\ j \in J,\ s \in S$$

Restricciones

1. Una posición por caja

$$\sum_{i \in I}^{l} \sum_{j \in J}^{l} \sum_{s \in S}^{h} x_{k,i,j,s} = 1\ \forall k \in K$$

2. Una caja por posición (restricción necesaria para pallet completo)

$$\sum_{k \in K}^{n} x_{k,i,j,s} = 1\ \forall i \in I,\ j \in J,\ s \in S$$

3. Orden de prioridades

$$\sum_{k \in K}^{n} p_k \cdot x_{k,\ i+1\ ,j,s} \leq \sum_{k \in K}^{n} p_k \cdot x_{k,i,j,s} \quad \forall i \in \{1,\ \ldots,\ l-1\},\ j \in J,\ s \in S$$

El lado izquierdo de la ecuación adquiere el valor de la caja (k) en una fila más cercana a la salida (i+1) y se compara con el lado derecho, el cual daría la prioridad de la caja actual (caja justo debajo de esa) y se asegura de que las prioridades pertenecientes a cajas más cercanas a la salida siempre sean menores.

4. Capacidad de carga volumétrica

$$\sum_{k' \in K}^{n} \sum_{s'=s+1}^{h} \omega_{k'} \cdot x_{k',\ i\ ,j,\ s'} \leq \sum_{k \in K}^{n} \gamma_k \cdot x_{k,i,j,s} \quad \forall s \in \{1,\ \ldots,\ l-1\},\ j \in J,\ i \in I$$

Esta restricción asegura que el peso de una caja por encima de la actual ($\omega_{k'} \cdot x_{k',\ i\ ,j,\ s'}$) nunca sea mayor a la capacidad de carga de la caja sobre la que está ($\gamma_k \cdot x_{k,i,j,s}$).

Formalización de la función objetivo

Ya que el coste total es acumulativo, y en una fila de 3 el coste total sería $Coste\ Total = 2 \cdot p3 + 1 \cdot p2 + 0p1$ lo cual es igual a (generalizando):

$$Coste\ Total = \sum_{k \in K}^{n} \sum_{i \in I}^{l} \sum_{j \in J}^{l} \sum_{s \in S}^{h} (i-1) \cdot p_k \cdot x_{k,i,j,s}$$

Nuestra función objetivo para este apartado es:

$$min\ Z = \frac{1}{n}(Coste\ Total)$$

Lo cual es igual a:

$$min\ Z = \frac{1}{n}\sum_{k\in K}^{n}\sum_{i\in I}^{l}\sum_{j\in J}^{l}\sum_{s\in S}^{h}(i-1)\cdot p_k \cdot x_{k,i,j,s}$$

Donde Z es el Coste Medio.

Parte 3: Optimización de pallets incompletos

Descripción del problema

El apartado 3 expande una vez más sobre el problema establecido en el apartado previo. Esta vez es el mismo problema que en el apartado 2 solo que los niveles de los pallets no tienen porqué estar completos. esto significa que la idea de cajas flotantes (cajas en niveles superiores sin cajas debajo) se convierte en un nuevo reto, además te tener en cuenta que las posiciones vacías tienen un coste de 0 y prioridad infinitamente grande lo cual deja nuestra función objetivo previa completamente obsoleta ya que se basaba en un pallet completo siempre. Nuestra solución utiliza una variable auxiliar binaria que facilita establecer los costes de qué cajas sumas basado en restricciones impuestas a esa variable en particular que no se podían hacer con nuestra variable de decisión original si queríamos conservar una función lineal.

Descripción del modelo

Parámetros

Se definen los siguientes **parámetros**:

- ➔ $P_{max} = max\ p_k$: valor máximo de prioridad para las cajas $k \in K$

Variables de decisión

Mantenemos $x_{k,i,j,s} \in \{0,\ 1\} \forall k \in K,\ i \in I,\ j \in J,\ s \in S$ del apartado anterior.

Añadimos la variable (binaria auxiliar) para cada caja *k* en posición (*i+1, j, s*) y :

$$y_{k,i,j,s} = \begin{cases} 1 \text{ si la caja } k \text{ ocupa la posición } (i+1,\ j,\ s) \text{ y posición posterior no está vacía,} \\ 0 \text{ en caso contrario} \end{cases}$$

Su dominio es:

$$y_{k,i,j,s} \in \{0,\ 1\} \forall k \in K,\ i \in \{1,\ l-1\},\ j \in J,\ s \in S$$

Restricciones (añadidas o cambiadas)

1. Máximo una caja por posición

$$\sum_{k\in K}^{n} x_{k,i,j,s} \leq 1\ \forall i \in I,\ j \in J,\ s \in S$$

2. Orden de prioridades (con Big-M siendo P_{max})

$$\sum_{k\in K}^{n} p_k \cdot x_{k,\,i+1\,,j,s} \leq \sum_{k\in K}^{n} p_k \cdot x_{k,i,j,s} + P_{max} \cdot \left(\sum_{k\in K}^{n} p_k \cdot x_{k,i,j,s}\right) \forall i \in \{1, \ldots, l-1\},\ j \in J,\ s \in S$$

La restricción de prioridades se le añade una suma con la técnica de Big-M para que si una caja $x_{k,i,j,s}$ está vacía se le considere una prioridad increíblemente grande (suma el valor de P_{max}) y si no está vacía se cancela ese sumatorio.

3. Sin Huecos entre cajas (Ocupación contigua)

$$\sum_{k\in K}^{n} x_{k,i,j,\,s} \leq \sum_{k\in K}^{n} x_{k,i+1,j,\,s}\ \forall i \in \{1, \ldots, l-1\},\ j \in J,\ s \in S$$

4. Sin cajas Flotantes

$$\sum_{k\in K}^{n} x_{k,i,j,\,s+1} \leq \sum_{k\in K}^{n} x_{k,i,j,\,s}\ \forall i \in I,\ j \in J,\ s \in \{1, \ldots, h-1\}$$

5. Linearización de la variable auxiliar ($y_{k,i,j,\,s}$)

Establecemos que las posiciones cuyos costes deberíamos contar son aquellas que no estén en posición de salida (tiene cajas delante) ni estén vacías ya que su coste igualará a cero. Además, consideramos que solo se le sumará el coste de una caja particular si esta tiene una caja detrás y no un espacio vacío, ya que no necesitamos pasar por esa caja para llegar a otra y por lo tanto su coste es solamente el de aquellas cajas por delante suyo.

a. Si la caja k no tiene una caja delante, $x_{k,\,i+1\,,j,\,s} = 0$, entonces $y_{k,i,j,\,s} = 0$.

$$y_{k,i,j,\,s} \leq x_{k,\,i+1\,,j,\,s}\ \forall i \in \{1, \ldots, l-1\},\ j \in J,\ s \in S$$

b. Si la caja k no tiene una caja detrás (ninguna posición), $\sum_{k'\in K}^{n} x_{k',\,i,j,s} = 0$, entonces $y_{k,i,j,\,s} = 0$.

$$y_{k,i,j,\,s} \leq \sum_{k'\in K}^{n} x_{k',\,i,j,s}\ \forall i \in \{1, \ldots, l-1\},\ j \in J,\ s \in S$$

c. Si la caja k tiene una caja delante, $x_{k,\,i+1\,,j,\,s} = 1$, **y también** entonces forzamos $y_{k,i,j,\,s} = 1$. ($y_{k,i,j,\,s} \geq a \wedge b$)

$$y_{k,i,j,\,s} \geq x_{k,\,i+1\,,j,\,s} + \sum_{k'\in K}^{n} x_{k',\,i,j,s} - 1\ \forall i \in \{1, \ldots, l-1\},\ j \in J,\ s \in S$$

Formalización de la función objetivo

Para este modelo solo contamos el coste de cajas que tengan una caja delante y detrás (o que sean las últimas de su fila). Usamos la variable $y_{k,i,j,s}$ que tiene valor de 1 si es una de esas cajas específicas y 0 en cualquier otro caso. Así evitamos sumar el coste de espacios vacíos ya que tienen coste = 0.

$$Coste\ Total = \sum_{j\in J}^{l}\sum_{s\in S}^{h}\sum_{k\in K}^{n}\sum_{i\in I}^{l-1} p_k \cdot y_{k,i,j,s}$$

$$min\ Z = \frac{1}{n}(Coste\ Total)$$

Lo cual es igual a:

$$min\ Z = \frac{1}{n}\sum_{k\in K}^{n}\sum_{i\in I}^{l}\sum_{j\in J}^{l}\sum_{s\in S}^{h} p_k \cdot y_{k,i,j,s}$$

Donde *Z* es el Coste Medio.

Parte 4: Optimización del montaje en una cantidad *arbitraria* de pallets incompletos.

Descripción del problema

En esta parte disponemos de un conjunto de *n* cajas, que pueden estar distribuidas en varios pallets, cada uno de lado *l* y con un máximo de *h* niveles. Continuamos con las condiciones de la parte 3 con la diferencia de que utilizar un pallet implica un **coste fijo de montaje**. Entonces, se debe determinar no solo la distribución, sino el número de pallets empleados.

Queremos finalmente minimizar el **coste total**. Formulamos un problema de **Programación Lineal Entera Mixta** (**PLEM**) (decisiones de colocación y utilización son binarias, variables auxiliares son continuas; todo realizado linealmente).

Descripción del modelo

Parámetros y conjuntos

Definimos:

- ➔ l: lado del pallet.
- ➔ h: máximo de niveles.
- ➔ n: número de cajas.
- ➔ p_k: prioridad numérica.
- ➔ w_k: peso.
- ➔ γ_k: capacidad de carga
- ➔ *a*: coste de montaje de cada pallet (denominado *P* en el enunciado y precio en el modelo).

Los conjuntos serán entonces:

$K = \{1, 2, \ldots, n\}$ $\qquad$ $S = \{1, 2, \ldots, h\}$

$B = \{1, 2, \ldots, n\}$ $\qquad$ $I = J = \{1, 2, \ldots, l\}$

Donde:

- ➔ *K*: conjunto de cajas.
- ➔ *B*: conjunto de pallets.
- ➔ *S*: conjunto de niveles.
- ➔ *I*: conjunto de filas.
- ➔ *J*: conjunto de columnas.

Variables de decisión

- ➔ $x_{kbsij} \in \{0, 1\}$: vale 1 si la caja k se coloca en el pallet b, nivel s, fila i y columna j.
- ➔ $y_b \in \{0, 1\}$: vale 1 si se utiliza el pallet b. En MathProg, es **usar[b]**.
- ➔ $P_{bsij} \geq 0$: prioridad numérica de la caja situada en la posición (b, s, i, j); toma valor 0 si está vacía. Es una variable auxiliar.
- ➔ $C_{bsij} \geq 0$: coste asociado a la caja de esa posición, o 0 si no hay caja. Segunda variablke auxiliar.

Restricciones

1. Prioridad asociada a cada posición

La variable auxiliar recoge la prioridad de la caja colocada; una posición vacía tiene valor auxiliar nulo:

$$p_{bsij} = \sum_{k \in K} p_k x_{kbsij} \quad \forall b, s, i, j$$

2. Coste de una posición ocupada

Formulación Big-M. Las cajas situadas delante corresponden a filas con índice mayor:

$$C_{bsij} \geq \sum_{r=i+1}^{l} P_{bsrj} - M\left(1 - \sum_{k \in K} x_{kbsij}\right), \quad i < l.$$

Al minimizar una variable $C_{bsij} \geq 0$, si la posición está ocupada se obtiene el coste de las cajas que tiene delante; si está vacía, la restricción permite coste 0 siempre que M sea suficientemente grande.

3 y 4. Asignación y ocupación

Cada caja se coloca exactamente una vez entre todos los pallets, y cada posición admite como máximo una caja:

$$\sum_{b,s,i,j} x_{kbsij} = 1 \quad \forall k; \qquad \sum_{k \in K} x_{kbsij} \leq 1 \quad \forall b, s, i, j.$$

5 y 6. Activación y desactivación de los pallets

Vinculamos la utilización de cada pallet con la existencia de cajas en él; un pallet vacío no genera coste fijo y cualquier pallet con alguna caja queda activado:

$$\sum_{k,s,i,j} x_{kbsij} \leq n y_b; \qquad y_b \leq \sum_{k,s,i,j} x_{kbsij} \quad \forall b.$$

7 y 8. Contigüidad hacia el acceso y orden de prioridades

Impedimos cajas detrás de huecos dentro de una misma columna y se asegura que las cajas más prioritarias (menor valor numérico) estén delante:

$$\sum_{k} x_{kbsij} \leq \sum_{k} x_{kbs,i+1,j}, \qquad i < l;$$

$$P_{bs,i+1,j} \leq P_{bsij} + M\left(1 - \sum_{k} x_{kbsij}\right), \qquad i < l.$$

9 y 10. Soporte vertical y resistencia

Una caja en un nivel superior requiere otra debajo; además, cada caja debe resistir todo el peso situado por encima de su misma posición horizontal:

$$\sum_{k} x_{kb,s+1,i,j} \leq \sum_{k} x_{kbsij}\,, \qquad s < h;$$

$$\sum_{t=s+1}^{h} \sum_{k} \omega_k\, x_{kbtij} \leq \sum_{k} \gamma_k\, x_{kbsij}, \qquad \forall b, s, i, j.$$

Formalización de la función objetivo

El objetivo es entonces minimizar el coste total. Para conseguir esto, planteamos esta función objetivo:

$$\min Z = \sum_{k} \sum_{i} \sum_{j} \sum_{s} c_{bsij} + a \sum_{b} c_b$$

El primer término mide el coste de disposición en todos los pallets; el segundo incorpora el coste de montaje una sola vez por cada pallet utilizado.

Parte 5: Análisis de resultados

PARTE 1 Y 2:

- Análisis de complejidad:

En la Parte 1, el número de variables aumenta con la cantidad de cajas y posiciones disponibles. El principal factor que determina el tamaño del modelo es el lado del pallet. Como existen l^2 cajas y l^2 posiciones, se generan l^4 variables binarias. Esta estructura entonces permite obtener una solución óptima mediante un procedimiento directo de ordenación, sin necesidad de explorar todas las asignaciones posibles.

En la Parte 2, se añade una tercera dimensión haciendo que el modelo sea tridimensional e incrementando las cajas y posiciones posibles ya que el total de cajas posibles pasa a ser $n = l^2 h$. Esto causa que el número de variables binarias aumente a $total = n\, l^2 h = l^4 h^2$. Además se le suman restricciones de carga y peso a cada caja lo cual convierte el problema en un modelo de programación lineal más complejo que el del apartado anterior.

- ¿Es posible resolver óptimamente alguna de las partes directamente sin usar Programación Lineal?

Para la Parte 1, Sí. La Parte 1 puede resolverse óptimamente mediante un algoritmo de ordenación:

1. Ordenar las cajas por prioridad numérica, de mayor a menor.
2. Colocar las primeras *l* cajas en la fila 1, la más alejada del acceso.
3. Colocar las siguientes *l* cajas en la fila 2 y continuar sucesivamente.
4. Calcular el coste medio de la disposición obtenida.

La ordenación requiere un tiempo del orden de $O(n \log n)$).

Para la Parte 2, No. Al estar cubriendo tres condiciones de posicionamiento; prioridad, carga y peso, no se puede resolver óptimamente con un algoritmo de ordenación.

- ¿Es posible que algunas instancias sean infactibles o no acotadas?

Para las entradas válidas definidas en el enunciado, siempre existe al menos una solución infactible en ambos modelos. Una entrada con un número de cajas diferente de l^2 no cumpliría las condiciones de esta parte y es rechazada. Respecto a la acotación, la función objetivo está acotada, ya que existe un número finito de asignaciones de las cajas y todas las prioridades son valores positivos finitos. Por tanto, para las entradas válidas, la Parte 1 y 2 son factibles y tienen un coste óptimo finito.

- ¿Es posible que haya más de una solución óptima?

Sí para ambos apartados. En la Parte 1 y 2 es posible encontrar varias soluciones óptimas, ya que intercambiar cajas dentro de una misma fila no cambia sus coeficientes en la función objetivo, siempre que se respete el orden de prioridades en las columnas y para la parte 2 es posible que existan dos cajas con la misma prioridad (entre columnas), peso y capacidad con lo cual haría que intercambiar sus posiciones no altere el valor óptimo obtenido.

- ¿Cuántas variables y restricciones obtienes en cada caso test? Explica cómo estas modificaciones afectan a la dificultad de resolución del problema resultante.

P1: TESTS

Instancia	Lado	Variables	Restricciones	Coste medio	Tiempo (s)
Mínimo	1	1	2	0	0,35
Lado 2	2	16	10	0,75	0,36
Lado 3	3	81	24	3	0,32
Lado 3, mismas prioridades	3	81	24	1	0,25
Ejemplo dado	3	81	24	3,666666667	0,24
Lado 4	4	256	44	7,75	0,23

Los resultados confirman que el número de variables aumenta de acuerdo con la expresión l^4, que entonces muestran que el número de variables crece más rápidamente que el de restricciones. También se observa que modificar únicamente las prioridades no altera las dimensiones del modelo.

P2: TESTS

Instancia	Qué Evalúa	Lado	Variables	Restricciones	Coste medio	Tiempo (s)
Flotación Extremo	Garantiza que el solver no coloque cajas flotando en el aire y exija siempre la presencia de una caja de apoyo en el nivel de debajo	2	144	38	0.5	0,173
Resistencia Factible	Demuestra que el solver es capaz de salvar la fragilidad de ciertos elementos	2	64	24	2,75	0,147

	reubicándolos en la capa superior del pallet.					
Resistencia Infactible	Acredita que GLPK identifica correctamente la imposibilidad física en lugar de ofrecer una solución errónea	2	-	-	Infactible	0,162
Peso Acumulado	Valida que el modelo considere el peso total combinado de todas las cajas superiores sobre la base y no únicamente el de la caja inmediatamente superior.	1	-	-	Infactible	0,135
Inversión propiedades	Asegura que las cajas de mayor prioridad queden siempre al frente, evitando que elementos de baja prioridad bloqueen la salida	2	16	10	0,5	0,159

- ¿Qué influye en el tiempo de resolución?

Para la Parte 1, el tiempo de resolución puede verse afectado por el lado del palet, ya que este determina el número de variables binarias y restricciones. Sin embargo, también influyen los valores de las prioridades y la facilidad con la que el solucionador puede encontrar y demostrar una solución óptima. No se observa un aumento progresivo del tiempo con el tamaño.

En la Parte 2, el tiempo de resolución depende sobre todo de la altura y valor de *l* del palet ya que son los factores que pueden hacer crecer exponencialmente no solo la complejidad del problema sino también el tiempo de búsqueda. El solver sin embargo puede cortar nodos que "sobrecarguen" el pallet si no tiene ninguna combinación posible en la que su capacidad de carga pueda con otra caja o viceversa, en estos casos el solver quita las cajas que sobrecargan el palet lo cual ayuda a no perder tiempo en resoluciones imposibles o problemáticas para el modelo.

- ¿Cuáles son los tamaños máximos que podéis resolver en cada caso?

Tomamos como "límite de tiempo" un límite artificial de 60 segundos aunque con palets donde l=3 se ha comprobado que los modelos tardan menos de 1 segundo en resolver el problema para ambas partes.

PARTE 3:

- Análisis de complejidad.

Para la parte 3 se añade al modelo anterior un factor de complejidad bastante alto, este siendo los palets incompletos que aumentan la complejidad de busqueda ya que el modelo no solo debe contar y llegar a cada posición del palet existente pero también debe decidir si contarle o no en la función objetivo basado en si es una posición vacía o está delante de una. Gracias a esta restricción se añade una variable binaria auxiliar que aumenta la complejidad del modelo al tener que asegurar iterativamente las celdas delanteras y traseras de cada caja.

- ¿Es posible resolver óptimamente alguna de las partes directamente sin usar Programación Lineal?

No. A partir de la integración de volumen a los palets no se puede resolver sin programación lineal y ahora con la necesidad de determinar las posiciones vacías es mucho más complejo.

- ¿Es posible que algunas instancias sean infactibles o no acotadas?

Al igual que en las partes anteriores las instancias infactibles siempre son posibles. Por ejemplo puedes tener un pallet incompleto con cajas demasiado pesadas pero demasiado frágiles (peso alto, capacidad de carga = 0) donde es imposible ordenar el palet sin romper la regla de flotación.

La actuación sigue siendo imposible ya que el modelo sigue acotado inferiormente (todos los valores han de ser positivos).

- ¿Es posible que haya más de una solución óptima?

Si, en este modelo existen varias soluciones óptimas ya que las columnas se podrían mover entre sí sin causar cambios en la función de coste medio óptima.

- ¿Cuántas variables y restricciones obtienes en cada caso test? Explica cómo estas modificaciones afectan a la dificultad de resolución del problema resultante.

P3: TESTS

Instancia	Qué Evalúa	Lado	Variables	Restricciones	Coste medio	Tiempo (s)
Minimización Perfecta	Habiendo espacio disponible, demuestra que el solver distribuye las cajas sin generar ningún bloqueo entre filas.	2	48	81	0	0,149
Bloqueo Forzado	Cuando la falta de espacio obliga a solapar cajas, confirma que el solver coloca la caja de menor prioridad ($P = 1$) en la posición de la salida.	2	18	34	0,3333	0,151
Lleno contra Incompleto	Permite comprobar cómo se coordinan la regla de flotación (llenar la base) con la minimización de visibilidad en el nivel superior (nivel superior incompleto).	2	16	94	0,2	0,213
Sacrificio Peso Coste	Verifica la capacidad del solver para equilibrar dos objetivos encontrados: no	2	48	81	0	0,197

	sobrecargar cajas frágiles vs. no empujar atrás cajas prioritarias.					
Subóptimo por Resistencia	La resistencia y peso están por encima de la función objetivo: el solver prefirió reubicar las cajas en una posición con mayor coste de bloqueo ($z = 1.25$) antes que colocarlas en una posición de menor coste pero que aplastará la caja inferior.	2	48	81	1,25	0,138

- ¿Qué influye en el tiempo de resolución?

En este caso sobre todo va a influir la densidad de ocupación en el palet al tiempo de resolución ya que por cada posición no vacía el solver debe evaluar ambas partes de la restricción de linealización de la variable binaria auxiliar, aumentando así el tiempo de procesamiento.

- ¿Cuáles son los tamaños máximos que podéis resolver en cada caso?

Seguimos poniendo el límite en 60 segundos pero en este caso con la nueva complejidad añadida los pallets incompletos que tengan un valor de hasta $l = 3$ con un número de cajas menor a 27 se podrá hacer en menos de 5 segundos, lo cual es un aumento en tiempo de procesamiento comparado con los apartados 1 y 2 lo cual demuestra el incremento de complejidad en el modelo.

PARTE 4:

- Análisis de complejidad.

Se trata de un problema de programación lineal entera mixta, con numerosas combinaciones posibles de asignación y apilamiento. El número de variables principales crece cuadráticamente con *n*, porque aumentar el número de cajas también aumenta el número de pallets candidatos. Asimismo, crece linealmente con *h* y cuadráticamente con *l*.

En general, su resolución exacta puede requerir una exploración combinatoria elevada, por lo que su dificultad aumenta rápidamente al crecer las dimensiones.

- ¿Es posible resolver óptimamente alguna de las partes directamente sin usar Programación Lineal?

Por lo general, no existe un procedimiento de ordenación equivalente que resuelva directamente todos los casos de la Parte 4 ya que no es sólo ordenar las cajas, sino decidir cuántos pallets utilizar, cómo repartir las cajas entre ellos y si es posible apilarlas.

- ¿Es posible que algunas instancias sean infactibles o no acotadas?

Esta parte admite una solución factible para las instancias habituales del problema, siempre que la constante Big-M sea suficientemente grande. En la implementación se mantiene la constante

(M=100000). Para los casos de prueba utilizados, su valor es suficiente. Si los datos fueran muy grandes con respecto a esta constante, las restricciones Big-M podrían excluir alguna disposición válida.

En cuanto a la acotación, la función objetivo suma los costes de disposición y los costes de preparación de los pallets utilizados. Ambos términos son no negativos, por lo que el valor objetivo no puede disminuir indefinidamente. Por tanto, no se esperan problemas de no acotación de la función objetivo en ninguna de las dos partes.

- ¿Es posible que haya más de una solución óptima?

Sí, pueden existir múltiples soluciones óptimas. Una causa puede ser que los pallets son equivalentes entre sí, o puede haber diferentes formas de distribuir las cajas o de colocarlas entre columnas sin modificar el coste de disposición ni el número de pallets preparados.

- ¿Cuántas variables y restricciones obtienes en cada caso test? Explica cómo estas modificaciones afectan a la dificultad de resolución del problema resultante.

P4: TESTS

Instancia	Configuración	Variables	Restricciones	Coste total	Tiempo (s)
Precio bajo	$l = 2$, $h = 1$, $precio = 2$	100	85	4	0,30
Precio alto	$l = 2$, $h = 1$, $precio = 4$	100	85	7	0,24
Dos niveles	$l = 2$, $h = 2$, $precio = 2$	196	173	2	0,36
Sin apilamiento	$l = 2$, $h = 2$, $precio = 2$, $\gamma = 0$	196	173	4	0,35
Ejemplo dado	$l = 2$, $h = 2$, $n = 6$ $precio = 3$	390	259	6	0,24

Estos resultados muestran que tanto las dimensiones como los valores de los parámetros afectan al problema, aunque no necesariamente de la misma forma.

- ¿Qué influye en el tiempo de resolución?

➔ Número de cajas n: aumentar la cantidad de cajas provoca un crecimiento cuadrático de las variables principales.
➔ Lado del pallet l: incrementa cuadráticamente el tamaño del modelo.
➔ Número de niveles h: aumenta las posiciones disponibles e introduce más posibilidades de apilamiento.
➔ Prioridades, pesos y capacidades: modifican las disposiciones factibles y pueden hacer más fácil o difícil encontrar el óptimo.
➔ Coste de preparación: cambia el equilibrio entre utilizar pocos pallets y reducir los costes de disposición.
➔ Simetrías entre pallets: existen muchas asignaciones equivalentes al intercambiar identificadores de pallets, lo que puede complicar la exploración de soluciones.

- ¿Cuáles son los tamaños máximos que podéis resolver en cada caso?